

P3405R0

Out-of-order designated initializers

Published Proposal, 2024-09-28

This version:

<http://wg21.link/P3405R0.html>

Author:

[Elias Kosunen](#)

Audience:

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper proposes relaxing the rules regarding designated initializers by allowing the initializers to appear out of order.

Table of Contents

1 Changelog

1.1 R0 (Pre-Wrocław, 10/2024)

2 Introduction

3 Motivation

4 Background

5 Proposal

5.1 (A): Trivially destructible types

- 5.2 (B): Initialization through temporaries
- 5.3 (C): Out-of-order list initialization
- 5.4 Messing with the order of construction and destruction
- 5.5 Summary

6 Implementation and wording

References

Informative References

§ 1. Changelog

§ 1.1. R0 (Pre-Wrocław, 10/2024)

Initial revision.

§ 2. Introduction

Designated initializers were introduced in C++20, by [\[P0329R4\]](#) (design in [\[P0329R0\]](#)), adapted from C. However, in C, designated initializers are arguably more powerful than in C++. This is due to some deliberate limitations placed on said syntax in C++. While there are valid technical reasons for these limitations, to this day, this remains as a common point of surprise, and is often cited by a pain point by the community at large.

```
// Example adapted from Annex C ([diff.dcl] p9)
struct A { int x, y; };
struct B { struct A a; }

// Valid C, valid C++
struct A a = {.x = 1, .y = 2};
// Valid C, invalid C++ (out-of-order initializers)
struct A a = {.y = 1, .x = 2};
// Valid C, invalid C++ (array initializers)
int arr[3] = {[1] = 5};
// Valid C, invalid C++ (nested initializers)
```

```
struct B b = {.a.x = 0};  
// Valid C, invalid C++ (mixed initializers)  
struct A a = {.x = 1, 2};
```

This paper proposes making some out-of-order designated initializers well-formed, while leaving the other omissions be.

```
struct A { int x, y; };  
// Valid C, proposed to be valid C++  
struct A a = {.y = 1, .x = 2};
```

§ 3. Motivation

As mentioned above, the fact that out-of-order designated initializers are not supported is surprising to users.

Using designated initializers in code increases readability. It makes it explicit which specific fields are being initialized, and with which values. Initializing a struct with multiple fields without designated initializers can be difficult to grok without external tooling support (like IDEs) or handcrafted comments.

Aggregates that are initialized with designated initializers are a common pattern for emulating named function parameters. However, the utility of this pattern is diminished by the fact that these "named parameters" have to appear in order. It's not difficult to imagine a function taking a number of named arguments, with the user having difficulty remembering the order they appear within the intermediary struct, which will just cause the compiler to yell at the user for no clear reason.

```
struct named_arguments {  
    int foo;  
    std::string option;  
    bool flag;  
    int bar;  
};  
  
void func(named_arguments);  
  
// `.bar` appears before `.flag`, error  
func({ .foo=123, .bar=456, .flag=true });
```

As a variant of the above, designated initializers can be used as a form of the builder pattern, where a resulting object is constructed based on the state of an intermediate (builder) object. These builders are often assigned state using chained member function calls. Crucially, this can be done in arbitrary order, while also assigning names to each of the values given to the builder, increasing legibility. Allowing out-of-order designated initializers would make simple aggregates usable as builders.

```
struct my_type;

// Before:
struct my_type_builder {
    my_type_builder() = default;

    my_type_builder & foo(int v) {
        assert(not foo_);
        foo_ = v;
        return *this;
    }
    my_type_builder & bar(std::string v) {
        assert(not bar_);
        bar_ = std::move(v);
        return *this;
    }

    my_type done() &&;

private:
    std::optional<int> foo_;
    std::optional<std::string> bar_;
};

auto val = my_type_builder{}
    .foo(123)
    .bar("Hello world!")
    .done();

// After:
struct my_type_builder {
    int foo;
    std::string bar;

    my_type done() &&;
};
```

```
auto val = my_type_builder{
    .foo = 123,
    .bar = "Hello world!"
}.done();
```

Take note, how in the "After"-example, initializing the same field multiple times is prevented statically by-construction, whereas runtime asserts (or some other error checking mechanisms) are required for the "Before"-case. Similarly, required fields could be implemented with a per-field wrapper class that doesn't have a default constructor, whereas runtime checking is again required here for the "Before"-case.

In addition, this is a point of incompatibility with C and C++. In an idealistic sense, it seems silly that C has a language feature, while C++ only has a cut-down version of that same feature. In the least, we should look into supporting this syntax for types for which C already supports it, i.e., POD types. Although, the actually relevant type category here seems to be whether a type has a trivial destructor: we don't need to require full PODness (which is also no longer a thing).

§ 4. Background

The core of the issue lies within the rules for the order of construction and destruction of objects. In C++, objects are constructed in the order they are declared, and destructed in the reverse order of construction. This guarantee holds for all types and constructs within the language. This is also somewhat present in C: the fields of a struct are initialized in order, while destructors are not a thing in C.

```
struct C {
    T a{};
    T b{};
};

void foo() {
    // c.a is constructed first, followed by c.b
    C c{};
    // c.b is destructed first, followed by c.a
}
```

In addition to the order of (de)initialization, there's also the issue of the order of evaluation for the initializers. Currently, with all *list-initialization* (including aggregate initialization, which includes designated initializers), the initializers are guaranteed to be evaluated in lexical order, i.e., in the order they

appear. This is different from C, where the order of evaluation of the initializers of an aggregate are unspecified.

[dcl.init.list] p4

Within the *initializer-list* of a *braced-init-list*, the *initializer-clauses*, including any that result from pack expansions ([temp.variadic]), are evaluated in the order in which they appear. That is, every value computation and side effect associated with a given *initializer-clause* is sequenced before every value computation and side effect associated with any *initializer-clause* that follows it in the comma-separated list of the *initializer-list*.

[Note 4: This evaluation ordering holds regardless of the semantics of the initialization; for example, it applies when the elements of the *initializer-list* are interpreted as arguments of a constructor call, even though ordinarily there are no sequencing constraints on the arguments of a call. — *end note*]

```
struct D {
    int a;
    int b;
};

void foo() {
    // `123` is evaluated first, followed by `456`
    D d{123, 456};
}
```

In summary, there's two different orderings to consider here:

- the order to evaluate the initializers
- the order of construction and destruction

Currently, for consistency with list-initialization, and with the rest of the language in general, these orderings are equivalent (= lexical / left-to-right, up-to-down). In order to support out of order designated initializers, either:

1. one or both of these orderings need to be made different from the rest of the language, or
2. intermediary temporaries need to be created for out-of-order fields.

Below, the options (A) and (C) do 1., while option (B) does 2, although option (A) has a trick up its sleeve.

§ 5. Proposal

§ 5.1. (A): Trivially destructible types

There's a category of types where achieving the objective of this proposal could be considered to be relatively easy. That category is types that have a trivial destructor (= trivially destructible types).

Trivial destructors do nothing, their behavior isn't observable, and they don't even need to be called on object destruction. Therefore, the order in which the fields are constructed doesn't actually matter, as the order of destruction is in no way observable.

```
struct D {  
    int a;  
    int b;  
};  
  
void foo() {  
    // Order of construction is d.b followed by d.a:  
    // lexical order in list-initialization is still respected  
    D d{.b = 456, .a = 123};  
    // Order of destruction doesn't matter (it isn't observable),  
    // because `D` is trivially destructible  
}
```

It should be noted, that because the members are still initialized in lexical order, the common pitfalls with member initializer lists in class constructors can be avoided: the order of field initialization here is predictable, unsurprising, and can be determined solely by looking at the initializer. Also, because in aggregate initialization, fields cannot be initialized using the values of other fields, changing the order of construction doesn't have an effect on the actual initialization.

§ 5.2. (B): Initialization through temporaries

For types that don't have a trivial destructor, the matter is slightly more complicated. If we want to both guarantee lexical order of evaluation for the initializers, and to maintain ordered construction and

destruction of fields, the only option is for the compiler to insert temporaries, and essentially perform the initialization in two stages.

```
struct E {
    std::string a;
    std::string b;
};

void foo() {
    E e{.b = "bbb", .a = "aaa"};

    // Equivalent to:
    std::string __b_tmp = "bbb";
    std::string __a_tmp = "aaa";
    E e{.a = std::move(__a_tmp), .b = std::move(__b_tmp)};
    // Initializers evaluated left-to-right,
    // fields constructed in order of declaration,
    // and destructed in reverse order
}
```

This would only be required for out-of-order initializers: existing code using ordered designated initializers would exhibit no change in behavior or generated code. It should be noted, that the trivial destructability of a single field doesn't matter here, what matters is whether the type as a whole has a trivial destructor.

This option obviously would require the fields to be move constructible. If that operation isn't cheap, it could add non-trivial amounts of overhead.

§ 5.2.1. Semantics of initialization through temporaries

The pattern showed above, i.e., initialize each out-of-order field by first constructing a temporary of the field type, and then move constructing the fields from these temporaries, only works in the simple case. References and cv-qualifiers cause complications. Let's go over each of the cases to refine the correct semantics.

```
struct F {
    T          value;
    T &        lvalue_ref;
    T const &  const_lvalue_ref;
    T &&       rvalue_ref;
```



```

    T const && const_rvalue_ref;
    // skipping `volatile`,
    // it'll fall out naturally once we handle `const` properly
};

void foo() {
    // Construct everything out-of-order to force creation of temporaries
    F f{
        .const_rvalue_ref = ...,
        .rvalue_ref = ...,
        .const_lvalue_ref = ...,
        .lvalue_ref = ...,
        .value = ...
    };

    // All the temporaries have the same type as the fields:
    // references and cv-qualifiers included
    T const && __const_rvalue_ref_tmp = ...;
    T && __rvalue_ref_tmp = ...;
    T const & __const_lvalue_ref_tmp = ...;
    T & __lvalue_ref_tmp = ...;
    T __value_tmp = ...;

    F f{
        // We want to move the value
        .value = static_cast<T &&>(__value_tmp),
        // The lvalue reference can be initialized directly
        .lvalue_ref = __lvalue_ref_tmp,
        // Likewise for the lvalue reference to const
        .const_lvalue_ref = __const_lvalue_ref_tmp,
        // `__rvalue_ref_tmp` is an lvalue,
        // we need to cast it to an rvalue again
        .rvalue_ref = static_cast<T &&>(__rvalue_ref_tmp),
        // Likewise for the rvalue reference to const
        .const_rvalue_ref = static_cast<T const &&>(__const_rvalue_ref_tmp)
    };
}

```

Since `std::move(foo)` is equivalent to

`static_cast<std::remove_reference_t<decltype(foo)> &&>(foo)`, in the example above, all the casts are actually equivalent to calls to `std::move`. `const` (and therefore, `volatile`) don't seem to have an effect here.

Therefore, the rule is as follows:

- Construct a temporary variable, of type `decltype(field)`, with the initializer given in the list-initialization
- If `decltype(field)` is an lvalue reference, initialize the field directly from the temporary,
- otherwise, initialize the field as-if by a call to `std::move`

Because of this, strictly speaking, we don't actually need to require the out-of-order fields to be move constructible. Only the fields of a non-reference type need to be move constructible. Admittedly, that's not a large difference in practice, given the general avoidance for references as non-static data members.

§ 5.3. (C): Out-of-order list initialization

An alternative to the option detailed above, would be to take a look at how member initializer lists of constructors do things. It is a semi-well known fact, that the order of member initializers in a constructor doesn't matter: the fields are always constructed in the order of declaration within the class.

```
struct G {  
    // In reality,  
    // a_ is constructed before b_,  
    // even though b_ appears first, here  
    G(int a, int b) : b_(b), a_(a) {}  
  
    int a_;  
    int b_;  
};
```

In C++11, the rules for the order of evaluation in an expression were tightened. Before that, the order of evaluation within aggregate initialization was actually unspecified, which is still the case in C. Since C++11, however, the rules have been clear: list-initialization is always performed left-to-right.

We could consider changing this, to sort of follow what member initializers in constructors do. The initializers could be evaluated in the order the members appear in the class. Since aggregate initializa-

tion requires for the type to be complete, the compiler has knowledge of the order of the members in the class, so this could be technically achievable.

```
void foo() {
    // `a` is the first member of the class, so its initializer is evaluated
    // and it's constructed first.
    // `b`'s initializer is evaluated and is constructed second.
    E e{.b = "bbb", .a = "aaa"};

    // Strictly equivalent to:
    E e{.a = "aaa", .b = "bbb"};
}
```

This ordering property of member initializers is however often cited as a common pitfall in C++, and can lead to potentially surprising behavior. Because designated initializers cannot refer to other (potentially uninitialized) fields of the aggregate, it wouldn't be as big of an issue here, but the altered order of evaluation could still be unexpected, especially if the fields themselves have sufficiently adventurous constructors. Because of this, this is given here as an alternative to option (B) [§ 5.2 \(B\): Initialization through temporaries](#) above, but not as the primary option.

```
void foo() {
    int i = 0;
    // d.a is initialized first, to 0
    // d.b is initialized second, to 1
    D d{.b = i++, .a = i++};
}

int global_state = 1;

struct H {
    struct Inner {
        Inner(int val) : value(global_state * val) {
            global_state *= 10;
        }

        int value;
    };

    Inner a;
    Inner b;
};
```

```

void bar() {
    // h.a initialized first, followed by h.b
    H h{.b = 456, .a = 123};
    // h.a is 123, h.b is 4560
}

```

It should be noted, that this wouldn't cause existing standard-compliant code to change its meaning. This change would only apply to out-of-order designated initializers, which are currently ill-formed.

(C) is the behavior currently implemented in Clang (as of v19.1.0), even when compiling C++, although it does issue a warning: [Compiler Explorer](#). GCC currently (as of v14.2) does not allow this code, even when compiling with extensions enabled (`-std=gnu++20`).

```

#include <iostream>

struct Field {
    Field(int i) : value(i) {
        std::cout << "Field(" << value << ")\n";
    }

    ~Field() {
        std::cout << "~Field(" << value << ")\n";
    }

    int value;
};

struct S {
    Field a;
    Field b;
};

int main() {
    int i = 0;
    S s = {.b = i++, .a = i++};
    // s.a.value is 0:
    // The initializer for .a is evaluated first
    //
    // Field(0) is called before Field(1):
    // fields are initialized in the order of declaration
    std::cout << ".a = " << s.a.value << '\n';
}

```

```

    std::cout << ".b = " << s.b.value << '\n';
}

// Output:
// Field(0)
// Field(1)
// .a = 0
// .b = 1
// ~Field(1)
// ~Field(0)

```

§ 5.4. Messing with the order of construction and destruction

This paper doesn't propose altering the order of destruction (except for types with trivial destructors, with option (A) § 5.1 (A): [Trivially destructible types](#)). Practically speaking, the order of destruction needs to be statically determinable, or otherwise some sort of hidden run-time tracking would be required. This would be insane, and is therefore not an option. The order of destruction needs to remain the same, i.e., non-static member variables need to be destructed in the reverse order they were declared.

```

// Not proposed!

E get_e();

void foo() {
    // Opaque function call,
    // how will we know in what order to destruct the non-static members?
    auto e = get_e();
}

```

There's a slightly less outlandish idea, where the order of destruction would remain the same, but the order of construction and destruction could be made to be out of sync with each other, by not always destructing objects in the reverse order of construction. However, this would break an extremely long-standing guarantee within the C++ object model, and for that reason, it's not reasonable.

```

// Not proposed!

void foo() {
    // `e.b` constructed before `e.a`:

```

```

// it appears first in the initializer,
// even though it's not first in the class
E e{.b = "bbb", .a = "aaa"};

// `e.b` destructed before `e.a`:
// fields are destructed in the reverse order of declaration
}

```

§ 5.5. Summary

To summarize, in addition to the status quo (no out-of-order designated initializers), the following options are presented by the proposal above:

(A) (for trivially destructible types)

+ (B) (for other aggregates)

[§ 5.1 \(A\): Trivially destructible types](#)

[§ 5.2 \(B\): Initialization through temporaries](#)

- Enables out-of-order designated initializers for struct types with a trivial destructor
- Enables out-of-order designated initializers for fields of a non-reference type that are move constructible
- Enables out-of-order designated initializers for all fields of a reference type
- Changes order of field construction when it's not observable: creates temporaries otherwise
- No changes to list-initialization (lexical order maintained)
- Destructors called in reverse order of construction, if observable

Only (A), only for trivially destructible types

[§ 5.1 \(A\): Trivially destructible types](#)

- Enables out-of-order designated initializers for struct types with a trivial destructor
 - No changes to list-initialization (lexical order maintained)
 - Changes order of field construction
 - Destructor calling order changed (not observable)
-

Only (B), extend to all aggregate types

[§ 5.2 \(B\): Initialization through temporaries](#)

- Enables out-of-order designated initializers for fields of a non-reference type that are move constructible
- Enables out-of-order designated initializers for all fields of a reference type
- Causes additional copying and temporaries (for out-of-order initializers)
- No changes to list-initialization (lexical order maintained)
- No changes to order of field construction or destruction

Only (C), extend to all aggregate types

[§ 5.3 \(C\): Out-of-order list initialization](#)

- Enables out-of-order designated initializers for all types
- Initializers no longer evaluated in lexical order
- No changes to order of field construction or destruction
- Implemented in Clang for C++

(B) and (C) are mutually exclusive. Only one of them is needed to support this proposal, and doing them both would be unnecessary, and only needlessly complicate things.

(A) and (C) in conjunction would be undesirable, as that would make the order of evaluation in designated initializers different based on the type of the object that's being initialized. There's no potential performance penalty associated with (C) (like with (B) with its temporaries), so doing an "optimization" with (A) is not needed.

The order of preference of the author is (A)+(B), (A), (B), (C). The author finds (A)+(B) the most attractive of the bunch, as it enables the use of out-of-order designated initializers for all aggregates, with no observable change in semantics (apart from the possible loss of copy elision through creation of temporaries). (C) does however have a clear advantage through the fact that it's already implemented in Clang.

It should be noted, that this is not really something we can change later, if and once it has been specified, as that would induce change in behavior in well-formed code. As this is currently specified to be ill-formed, we can choose the behavior we want quite freely.

Below is a table comparing these options, but organized in a slightly different manner:

	(A)+ (B)	(A)	(B)	(C)
Out-of-order designated initializers for all types	⚠ 1 or 2	⚠ 2	⚠ 1	✓
Maintains order of destruction of fields as the reverse order of declaration	✓	✓	✓	✓
Maintains order of destruction of fields as the reverse order of construction	✓ ³	✓ ³	✓	✓
Maintains order of construction as the order of declaration	✓ ³	✓ ³	✓	✓
Maintains lexical order of evaluation for list-initialization	✓	✓	✓	✗
Never creates temporaries for out-of-order initializers	⚠ ⁴	✓	✗	✓
Implemented in Clang	✗	✗	✗	✓ ⁵

¹: As long as all out-of-order fields of non-reference type are move constructible

²: As long as the whole structure is trivially destructible

³: Changed for trivially destructible types, which is not observable

⁴: Only for non-trivially destructible types

⁵: As an extension: currently issues a warning in C++

§ 6. Implementation and wording

As of now, this paper specifically hasn't been implemented, and no wording is provided. Option (C) § 5.3 (C): [Out-of-order list initialization](#) is implemented in Clang as an extension. This paper is intended to explore the design space, and to gauge potential interest in the feature.

§ References

§ Informative References

[P0329R0]

Tim Shen, Richard Smith, Zhihao Yuan, Chandler Carruth. *Designated Initialization*. 9 May 2016. URL: <https://wg21.link/p0329r0>

[P0329R4]

Tim Shen, Richard Smith. *Designated Initialization Wording*. 12 July 2017. URL:
<https://wg21.link/p0329r4>